AUTOMATED SECURITY ASSESSMENT & REPORTING PIPELINE

คู่มือการใช้งาน VulnCompass

ติดตั้ง · ใช้งานผ่านหน้าเว็บ · สแกนทั้ง 4 แบบ (SAST, Dependency, Container, DAST)
อ่านผลและรายงาน · ใช้ผ่าน CLI, API และ CI/CD · แก้ปัญหาที่พบบ่อย

> เวอร์ชัน: **1.0.2**
> GitHub: `github.com/jjjecop/security-pipe-ine` (public, MIT License)
> Docker image: `ghcr.io/jjjecop/vulncompass` และ `ghcr.io/jjjecop/vulncompass-web`
> อัปเดตล่าสุด: 8 พฤศจิกายน 2026

สารบัญ

1. VulnCompass คืออะไร	ภาพรวม, การสแกน 4 แบบ, ใช้งานได้ 3 ทาง
2. ติดตั้ง	แบบ Docker (แนะนำ) และแบบรันบนเครื่อง · เลือกแบบไหนดี
3. เตรียมก่อนสแกนครั้งแรก	ผู้ใช้, โฟลเดอร์และเว็บที่อนุญาตให้สแกน, สร้างโปรเจกต์แรก
4. ใช้งานผ่านหน้าเว็บ	login, Dashboard, หน้าการสแกน, ดาวน์โหลดรายงาน
5. การสแกนทั้ง 4 แบบ	SAST · Dependency · Container · DAST · สแกนทุกแบบพร้อมกัน
6. อ่านผลและรายงาน	ระดับความรุนแรง, risk score, EPSS, OWASP, สถานะ open/fixed
7. ใช้ผ่าน CLI	ตัวเลือกทั้งหมดของ `run_pipeline.py`
8. ใช้ใน CI/CD	security gate ด้วย `--fail-on`
9. ใช้ผ่าน API	API key และตัวอย่างคำสั่ง
10. จัดการผู้ใช้และดูแลระบบ	เพิ่ม/ปิดบัญชี, หยุดและเปิดระบบ, backup
11. แก้ปัญหาที่พบบ่อย	ข้อความ error และวิธีแก้
12. ใช้งานอย่างปลอดภัย	สิ่งที่ต้องระวังเมื่อใช้เครื่องมือสแกน

1. VulnCompass คืออะไร

VulnCompass เป็นระบบที่**รวมเครื่องมือหาช่องโหว่ 4 ตัวไว้ในที่เดียว** สั่งสแกนครั้งเดียวแล้วได้ผลเป็นรายการเดียวที่ตัดตัวซ้ำแล้ว เรียงตามความเสี่ยงจริง ติดตามว่าช่องโหว่ไหนแก้แล้ว และออกรายงาน PDF ภาษาไทยพร้อมคำแนะนำการแก้ไข

การสแกน 4 แบบ

แบบ	เครื่องมือ	สแกนอะไร	ตัวอย่างสิ่งที่เจอ
SAST	Semgrep	source code (อ่านโค้ดโดยไม่ต้องรัน)	SQL injection, ใช้ `eval()` กับข้อมูลจากผู้ใช้, รหัสผ่านเขียนตายตัวในโค้ด
Dependency	Trivy `fs`	library ที่โปรเจกต์ใช้ (อ่านจากไฟล์ lock)	library เวอร์ชันเก่าที่มี CVE เช่น `minimist` prototype pollution
Container	Trivy `image`	Docker image (ทั้ง package ของระบบและ library ข้างใน)	OpenSSL, zlib หรือ package ของ Alpine/Debian ที่มีช่องโหว่
DAST	OWASP ZAP	เว็บที่**รันอยู่จริง** (ทดสอบจากภายนอกเหมือนผู้ใช้)	ไม่มี security header, cookie ไม่มี `HttpOnly`, เปิดเผยข้อมูลเวอร์ชัน server

หลังสแกนเสร็จ ระบบทำอะไรต่อ

1. **แปลงผล**ของทุกเครื่องมือให้อยู่ในรูปแบบเดียวกัน
2. **ตัดตัวซ้ำ** ทั้งในรอบเดียวกันและข้ามรอบสแกน
3. **ติดตามสถานะ**: ช่องโหว่ที่เคยเจอ แต่สแกนรอบใหม่แล้วไม่เจอ จะถูกเปลี่ยนเป็น `fixed` ให้อัตโนมัติ
4. **เพิ่มข้อมูล**: จับคู่ CWE กับหมวด OWASP Top 10, ดึงคะแนน EPSS (โอกาสถูกโจมตีจริง) และคำแนะนำการแก้ไข
5. **คิดคะแนนความเสี่ยง** 0–10 จากระดับความรุนแรง, CVSS และ EPSS
6. **ออกรายงาน** เป็น PDF, HTML และ dashboard

ใช้งานได้ 3 ทาง

ทาง	เหมาะกับ	ดูบทที่
หน้าเว็บ	ใช้งานทั่วไป: ดูภาพรวม, สั่งสแกน, ดาวน์โหลดรายงาน	4 และ 5
CLI	สั่งจาก terminal หรือ script	5 และ 7
CI/CD	ตรวจอัตโนมัติทุกครั้งที่ push และให้ build ไม่ผ่านถ้าเจอช่องโหว่ร้ายแรง	8

ส่วนประกอบของระบบ



API ไม่สแกนเอง แต่ใส่งานลงคิว แล้ว **worker** เป็นผู้หยิบงานไปสแกน หน้าเว็บจึงตอบทันทีแม้การสแกนจะใช้เวลาหลายนาที

2. ติดตั้ง

ติดตั้งได้ 2 แบบ ทั้งสองแบบใช้ฐานข้อมูลร่วมกันได้ จึงเปิดสลับกันได้ภายหลัง

เลือกแบบไหนดี

	แบบ A: Docker (แนะนำ)	**แบบ B: รันบนเครื่อง**
ต้องติดตั้ง	Docker Desktop อย่างเดียว	Python 3.14, Node 24, Docker Desktop, Trivy, GTK3 runtime
เปิดระบบ	คำสั่งเดียว	เปิด 3 หน้าต่าง (API, หน้าเว็บ, worker)
หน้าเว็บ	`http://localhost:8080`	`http://localhost:5173`
SAST	**ได้**	**ได้**
Dependency	**ได้**	**ได้**
Container	**ได้ เฉพาะ image ที่อยู่บน registry**	**ได้ ทั้งบน registry และ image ที่ build ไว้ในเครื่อง**
DAST	**ไม่ได้**	**ได้**
path ของโฟลเดอร์ที่สแกน	`/targets/ชื่อโฟลเดอร์`	`C:\Allprojects\Targets\ชื่อโฟลเดอร์`

> **ทำไมแบบ Docker สแกน DAST ไม่ได้:** ZAP ต้องรันเป็น container แยก ถ้าจะให้ worker ใน Docker สั่งเปิด container ได้ ต้องให้สิทธิ์ `docker.sock` ซึ่งเท่ากับให้สิทธิ์ root ทั้งเครื่อง ระบบจึงตั้งใจไม่ทำ ถ้าต้องสแกน DAST ให้ใช้แบบ B

แบบ A: Docker

ขั้นที่ 1: สร้างไฟล์ตั้งค่า `.env`

```
git clone https://github.com/jjjecop/security-pipe-ine.git
cd security-pipe-ine
Copy-Item .env.example .env
code .env
```

เติมค่าที่ว่างอยู่ ค่าลับทุกตัวให้สุ่มขึ้นมาใหม่ ห้ามใช้คำที่เดาง่าย คำสั่งนี้สุ่มให้ 1 ค่า (รันซ้ำเพื่อสุ่มค่าถัดไป):

```
python -c "import secrets; print(secrets.token_hex(24))"
```

ค่าใน `.env`	**ใช้ทำอะไร**	**จำเป็นไหม**
`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`	บัญชีของฐานข้อมูล	จำเป็น
`REDIS_PASSWORD`	รหัสผ่านของ Redis (ถ้าไม่ตั้ง Docker จะไม่ยอมเปิด)	จำเป็น
`SESSION_SECRET`	ใช้เซ็น cookie ของการ login (ถ้าไม่ตั้ง ทุกคนต้อง login ใหม่ทุกครั้งที่ restart)	ควรตั้ง
`API_KEY`	ใช้เรียก API จาก script หรือ CI (บทที่ 9)	ถ้าใช้ API
`GEMINI_API_KEY`	ให้ AI เขียนคำแนะนำการแก้ไขให้ CWE ที่ยังไม่มีคำแนะนำ	ไม่จำเป็น
`SCAN_ALLOWED_REPO_ROOTS`, `SCAN_ALLOWED_HOSTS`	โฟลเดอร์และเว็บที่อนุญาตให้สแกน (บทที่ 3)	จำเป็น
`TARGETS_DIR`	(Docker) โฟลเดอร์บนเครื่องที่จะให้เห็นเป็น `/targets` ค่าเริ่มต้นคือ `../Targets`	ไม่จำเป็น

> ห้าม commit หรือส่งไฟล์ `.env` ให้ใคร (อยู่ใน `.gitignore` แล้ว)

ขั้นที่ 2: เปิดระบบ

เปิด Docker Desktop ให้ขึ้น **Engine running** แล้วรัน (ครั้งแรกใช้เวลาประมาณ 5–10 นาที):

```
docker compose --profile app up -d --build
```

ตรวจว่าทุก service ทำงาน: `api`, `worker`, `web`, `redis` ต้องเป็น `Up`, `postgres` เป็น `healthy` และ `migrate` เป็น `Exited (0)` (สร้างตารางเสร็จแล้วจบเอง ถือว่าปกติ)

```
docker compose --profile app ps -a
```

ขั้นที่ 3: สร้างผู้ใช้คนแรก แล้วเปิดหน้าเว็บ

```
docker compose exec api python manage_users.py create admin
```

ตั้งรหัสผ่านอย่างน้อย 12 ตัวอักษร แล้วเปิด **`http://localhost:8080`**

แบบ B: รันบนเครื่อง (Windows)

ติดตั้งครั้งแรก

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements-dev.txt
Copy-Item .env.example .env          # แล้วเติมค่าตามตารางในแบบ A
docker compose up -d                 # เปิดแค่ PostgreSQL กับ Redis
python init_db.py                    # สร้างตาราง หรืออัปเดตฐานข้อมูลเดิม
python seed_cwe_mapping.py           # ข้อมูล CWE → OWASP และคำแนะนำการแก้ไข
cd frontend; npm ci; cd ..
python manage_users.py create admin
```

เปิดระบบทุกครั้ง (3 หน้าต่าง terminal)

หน้าต่าง	คำสั่ง	หมายเหตุ
1. API	`uvicorn main:app --reload`	ที่ `127.0.0.1:8000`
2. หน้าเว็บ	`cd frontend` แล้ว `npm run dev`	เปิด `http://localhost:5173`
3. worker	`python worker.py`	ต้องเปิดไว้ งานสแกนถึงจะทำงาน

> **ห้ามเปิด worker บนเครื่องพร้อมกับ worker ใน Docker** ทั้งสองตัวตั้งใจให้ใช้ Redis คนละ DB เพื่อให้แต่ละตัวหยิบเฉพาะงานของตัวเอง แต่ถ้าเปิดพร้อมกันแล้วสั่งสแกนสลับหน้าเว็บ จะงงว่างานไปอยู่ที่ไหน

ทดสอบว่าติดตั้งถูกต้อง

เช็ก	ควรเห็น
เปิด `/api/health` (เช่น `http://localhost:8080/api/health`)	`{"status":"ok","database":"ok"}`
login ที่หน้าเว็บ	เข้าหน้า Dashboard ได้
(แบบ B) `pytest -q`	`128 passed`

3. เตรียมก่อนสแกนครั้งแรก

3.1 วางโค้ดที่จะสแกนไว้ในโฟลเดอร์ที่อนุญาต

เพื่อความปลอดภัย ระบบสแกนได้**เฉพาะโฟลเดอร์ที่ตั้งไว้ใน allowlist** เท่านั้น ใครสั่งให้สแกนโฟลเดอร์อื่นในเครื่องจะถูกปฏิเสธ (ป้องกันการใช้ระบบไปอ่านไฟล์อื่น เช่น `C:\Users`)

	แบบ A: Docker	**แบบ B: บนเครื่อง**
โฟลเดอร์บนเครื่อง	`C:\Allprojects\Targets` (หรือค่าของ `TARGETS_DIR`)	ค่าของ `SCAN_ALLOWED_REPO_ROOTS` เช่น `C:\Allprojects\Targets`
path ที่พิมพ์ในหน้าเว็บ	`/targets/NodeGoat`	`C:\Allprojects\Targets\NodeGoat`
สิทธิ์	อ่านอย่างเดียว ระบบแก้ไฟล์ของคุณไม่ได้	ตามสิทธิ์ของ user ที่รัน worker

ถ้ามีหลายโฟลเดอร์ (แบบ B) คั่นด้วย `;` เช่น `SCAN_ALLOWED_REPO_ROOTS=C:\Allprojects\Targets;D:\Work`

3.2 ตั้งเว็บที่อนุญาตให้สแกน DAST

`SCAN_ALLOWED_HOSTS` คือรายชื่อ host ที่ DAST ยิงได้ คั่นด้วย `,` ค่าเริ่มต้นคือ `host.docker.internal,localhost`
ใส่เฉพาะเว็บที่**คุณเป็นเจ้าของหรือได้รับอนุญาตให้ทดสอบ**เท่านั้น

3.3 สร้างโปรเจกต์แรก

หน้าการสแกนบนเว็บเลือกได้เฉพาะโปรเจกต์ที่มีอยู่แล้ว ถ้ายังไม่มีเลยจะขึ้นว่า **"ยังไม่มีโปรเจกต์"**
โปรเจกต์จะถูกสร้างอัตโนมัติตอนสแกนครั้งแรกผ่าน CLI หรือ API ให้สั่งสแกนครั้งแรกแบบนี้:

```
# แบบ A (Docker): สแกนผ่าน worker ใน container
docker compose exec worker python run_pipeline.py --project NodeGoat --repo /targets/NodeGoat --scans sast

# แบบ B (บนเครื่อง)
python run_pipeline.py --project NodeGoat --repo C:\Allprojects\Targets\NodeGoat --scans sast
```

หลังจากนั้นโปรเจกต์ `NodeGoat` จะขึ้นในหน้าเว็บ และสั่งสแกนรอบต่อๆ ไปจากหน้าเว็บได้เลย

> **ชื่อโปรเจกต์** ใช้ได้เฉพาะ A–Z, a–z, 0–9, `_`, `.`, `-` ยาวไม่เกิน 100 ตัว ผลของแต่ละโปรเจกต์แยกกันทั้งหมด ถ้าใช้ชื่อเดิม ระบบจะเทียบกับผลรอบก่อนเพื่อหาว่าอะไรแก้แล้ว

4. ใช้งานผ่านหน้าเว็บ

4.1 Login

ใส่ username และรหัสผ่านที่สร้างไว้ ถ้าใส่ผิด 5 ครั้ง จะถูกล็อกจนครบ 15 นาทีนับจากครั้งแรกที่ผิด (ป้องกันการเดารหัส) และกด**ออกจากระบบ** แล้ว cookie เดิมจะใช้ไม่ได้ทันที

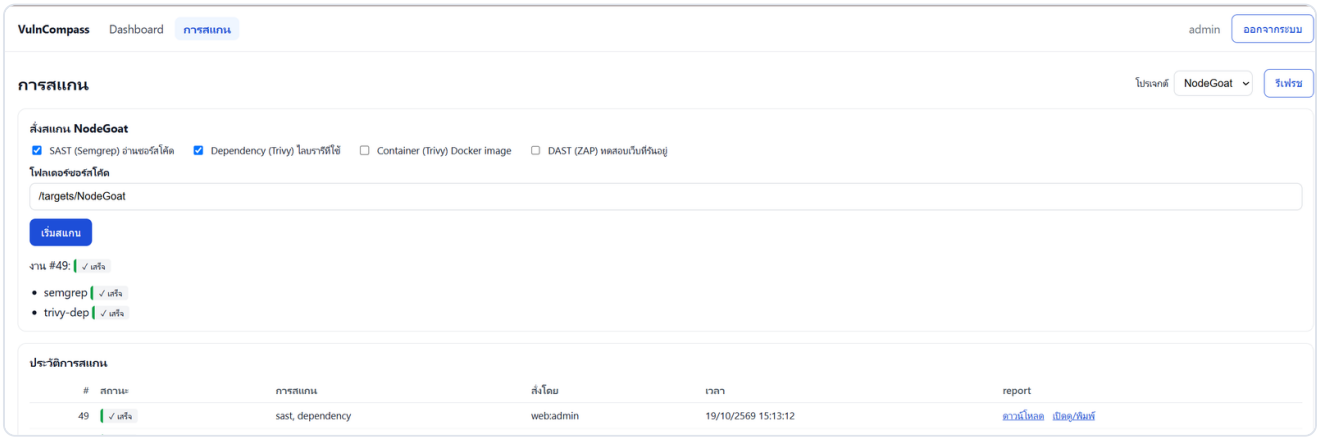
4.2 Dashboard



หน้า Dashboard ของโปรเจกต์ NodeGoat

ส่วน	ใช้ทำอะไร
ช่องเลือกโปรเจกต์ (ขวาบน)	สลับโปรเจกต์ ตัวเลขในวงเล็บคือจำนวนช่องโหว่ที่ยังเปิดอยู่
การ์ด 5 ระดับ	จำนวนช่องโหว่ที่ยังเปิดอยู่ แยกตาม วิกฤต / สูง / ปานกลาง / ต่ำ / ข้อมูล
กราฟแยกตามเครื่องมือ	ดูว่าช่องโหว่ส่วนใหญ่มาจากการสแกนแบบไหน
ตารางรายการช่องโหว่	เรียงตาม risk score จากมากไปน้อย ค้นหาได้ด้วยชื่อ, CVE, CWE, ไฟล์ หรือ URL และกรองตามความรุนแรงหรือเครื่องมือ

4.3 หน้าการสแกน



หน้าการสแกน: ฟอร์มสั่งสแกน สถานะของงานล่าสุด และประวัติการสแกน

1. เลือกโปรเจกต์ที่ช่องขวาบน
2. ติ๊กแบบการสแกนที่ต้องการ (ค่าเริ่มต้นคือ SAST + Dependency) ช่องกรอกข้อมูลจะเปลี่ยนตามแบบที่ติ๊ก:
   - SAST / Dependency → **โฟลเดอร์ซอร์สโค้ด**
   - Container → **Docker image**
   - DAST → **URL ของเว็บ**
3. กด **เริ่มสแกน** สถานะจะอัปเดตเองทุก 2 วินาที ไม่ต้องกด refresh
4. เมื่อเสร็จ ตารางประวัติจะมีลิงก์ **ดาวน์โหลด** (ไฟล์ PDF) และ **เปิดดู/พิมพ์** (เปิด PDF ในแท็บใหม่ แล้วสั่งพิมพ์ได้)

สถานะ	ความหมาย
⏳ รอคิว	อยู่ในคิว รอ worker หยิบไปทำ (ถ้าค้างนาน ให้เช็กว่า worker เปิดอยู่)
⟳ กำลังสแกน	worker กำลังสแกน
✓ เสร็จ	ทุกการสแกนสำเร็จ และสร้างรายงานแล้ว
⚠ เสร็จบางส่วน	บางเครื่องมือล้มเหลว (เช่น DAST เว็บปลายทางปิดอยู่) ผลของเครื่องมือที่สำเร็จยังบันทึกไว้ครบ
✕ ล้มเหลว	ทุกเครื่องมือล้มเหลว หรือ worker ถูกปิดระหว่างสแกน

> ถ้า worker ถูกปิดกลางคัน (เช่น ปิดหน้าต่าง) งานนั้นจะไม่ค้างเป็น "กำลังสแกน" ตลอดไป เปิด worker ใหม่แล้วระบบจะเปลี่ยนงานนั้นเป็น "ล้มเหลว" ให้เอง แล้วค่อยสั่งสแกนใหม่

5. การสแกนทั้ง 4 แบบ

ทุกแบบสั่งได้ทั้งจากหน้าเว็บ (บทที่ 4.3) และ CLI ตัวอย่างในบทนี้ใช้ NodeGoat ซึ่งเป็นเว็บที่ OWASP ทำขึ้นให้มีช่องโหว่ไว้ฝึก คำสั่ง CLI แบบ B รันที่โฟลเดอร์โปรเจกต์โดยเปิด venv ไว้ ส่วนแบบ A ให้ขึ้นต้นด้วย `docker compose exec worker`

1 SAST: สแกนซอร์สโค้ดด้วย Semgrep

ใช้ทำอะไร: อ่าน source code แล้วหารูปแบบที่เป็นช่องโหว่ **ไม่ต้องรันโปรแกรม** จึงสแกนได้ตั้งแต่ยังเขียนโค้ดไม่เสร็จ รองรับหลายภาษา เช่น JavaScript, TypeScript, Python, Java, Go, PHP

ต้องใส่: โฟลเดอร์ซอร์สโค้ด (ต้องอยู่ในโฟลเดอร์ที่อนุญาต บทที่ 3.1)

หน้าเว็บ: ติ๊ก **SAST (Semgrep)** → ใส่ `/targets/NodeGoat` (Docker) หรือ `C:\Allprojects\Targets\NodeGoat` (บนเครื่อง) → เริ่มสแกน

```
python run_pipeline.py --project NodeGoat --repo C:\Allprojects\Targets\NodeGoat --scans sast
```

ข้อควรรู้

- ใช้กฎแบบ `--config auto` ของ Semgrep ซึ่งดาวน์โหลดกฎล่าสุดจาก `semgrep.dev` ทุกครั้ง **ต้องต่ออินเทอร์เน็ต**
- ระบบจำช่องโหว่จากโค้ดที่ match ไม่ใช่เลขบรรทัด ถ้าเพิ่มโค้ดด้านบนจนบรรทัดเลื่อน จะยังนับเป็นช่องโหว่ตัวเดิม ไม่ขึ้นซ้ำ
- ผลในรายงานบอก **ไฟล์และบรรทัด** ที่เจอ

2 Dependency: สแกน library ที่โปรเจกต์ใช้ด้วย Trivy

ใช้ทำอะไร: อ่านไฟล์รายการ library (เช่น `package-lock.json`, `requirements.txt`, `pom.xml`, `go.sum`) แล้วเทียบกับฐานข้อมูลช่องโหว่ว่า library เวอร์ชันที่ใช้อยู่มี CVE ไหม และควรอัปเกรดเป็นเวอร์ชันอะไร

ต้องใส่: โฟลเดอร์ซอร์สโค้ดเดียวกับ SAST

หน้าเว็บ: ติ๊ก **Dependency (Trivy)** → ใส่โฟลเดอร์ → เริ่มสแกน (ติ๊กคู่กับ SAST ได้ ใช้โฟลเดอร์เดียวกัน)

```
python run_pipeline.py --project NodeGoat --repo C:\Allprojects\Targets\NodeGoat --scans dependency
```

ข้อควรรู้

- ครั้งแรก Trivy ต้องดาวน์โหลดฐานข้อมูลช่องโหว่ (ประมาณ 130 MB, 1–2 นาที) ครั้งต่อไปใช้ของเดิมที่เก็บไว้
- ต้องมีไฟล์ lock ในโฟลเดอร์ ถ้าโปรเจกต์ JavaScript ยังไม่เคย `npm install` อาจไม่มี `package-lock.json` และจะไม่เจออะไร
- ผลในรายงานบอกชื่อ package, **เวอร์ชันที่ใช้อยู่** และ**เวอร์ชันที่แก้แล้ว**

3 Container: สแกน Docker image ด้วย Trivy

ใช้ทำอะไร: ตรวจทุกอย่างที่อยู่ใน image ทั้ง package ของระบบปฏิบัติการ (Alpine, Debian, Ubuntu) และ library ของแอป เหมาะกับตรวจ image ก่อนนำขึ้น server

ต้องใส่: ชื่อ image เช่น `nginx:1.30.6-alpine`, `bkimminich/juice-shop`, `nodegoat:latest`

หน้าเว็บ: ติ๊ก **Container (Trivy)** → ใส่ชื่อ image ในช่อง **Docker image** → เริ่มสแกน

```
python run_pipeline.py --project NodeGoat --image nodegoat:latest --scans container
```

image แบบไหน	แบบ A: Docker	แบบ B: บนเครื่อง
อยู่บน registry (Docker Hub, ghcr.io)	**ได้**	**ได้**
build ไว้ในเครื่อง ไม่ได้ push ขึ้น registry (เช่น `nodegoat:latest`)	**ไม่ได้**	**ได้ (Trivy อ่านจาก Docker Desktop)**

ข้อควรรู้

- ชื่อ image ใช้ได้เฉพาะตัวอักษร ตัวเลข และ `. _ / : @ -` และห้ามขึ้นต้นด้วย `-` (กันไม่ให้ Trivy อ่านเป็นตัวเลือกของคำสั่ง)
- ควรระบุ tag ให้ชัด เช่น `:1.30.6-alpine` ถ้าไม่ระบุ จะได้ `:latest` ซึ่งเปลี่ยนไปเรื่อยๆ ผลแต่ละรอบจึงเทียบกันยาก
- CVE เดียวกันที่เจอทั้งใน Dependency และ Container จะแสดงในรายงานครั้งเดียว พร้อมระบุทั้งสองเครื่องมือ

4 DAST: ทดสอบเว็บที่รันอยู่จริงด้วย OWASP ZAP

ใช้ทำอะไร: เข้าเว็บจริงเหมือนผู้ใช้ (หรือผู้โจมตี) ไล่ดูทุกหน้า แล้วตรวจสิ่งที่เห็นได้จากภายนอก เช่น security header, การตั้งค่า cookie, ข้อมูลที่เผยออกมา ช่องโหว่แบบนี้ SAST มักหาไม่เจอ เพราะขึ้นกับการตั้งค่า server ไม่ใช่โค้ด

ใช้ได้เฉพาะแบบ B (worker บนเครื่อง) และต้องเปิด Docker Desktop ไว้ เพราะ ZAP รันเป็น container (`zaproxy/zap-stable`)

ขั้นตอน

1. **เปิดเว็บเป้าหมายให้ทำงานก่อน** เช่น NodeGoat: เข้าโฟลเดอร์ NodeGoat แล้ว `docker compose up -d` (เปิดที่ port 4000)
2. เช็กว่าเปิดเว็บ `http://localhost:4000` ในเบราว์เซอร์ได้
3. ที่หน้าเว็บ VulnCompass ติ๊ก **DAST (ZAP)** → ใส่ URL เป็น **`http://host.docker.internal:4000`** → เริ่มสแกน

```
python run_pipeline.py --project NodeGoat --url http://host.docker.internal:4000 --scans dast
```

ข้อควรรู้

- **ใช้ `host.docker.internal` แทน `localhost`:** ZAP รันอยู่ใน container ถ้าใส่ `localhost` ZAP จะไปหาเว็บใน container ของตัวเอง ซึ่งไม่มี
- host ของ URL ต้องอยู่ใน `SCAN_ALLOWED_HOSTS` และ URL ต้องเป็น `http` หรือ `https` เท่านั้น
- ใช้ **ZAP baseline scan** ซึ่งดูอย่างเดียว ไม่ส่งข้อมูลโจมตีจริง จึงปลอดภัยกับเว็บที่ใช้งานอยู่ แต่ใช้เวลาหลายนาที
- ครั้งแรก Docker ต้องดาวน์โหลด image ของ ZAP (ประมาณ 3.7 GB)
- ถ้าเว็บเป้าหมายปิดอยู่ DAST จะล้มเหลว และงานจะเป็น "เสร็จบางส่วน" ถ้าสแกนแบบอื่นด้วย

> ทดสอบเฉพาะเว็บที่คุณเป็นเจ้าของ หรือได้รับอนุญาตเป็นลายลักษณ์อักษรเท่านั้น
> การสแกนเว็บของคนอื่นโดยไม่ได้รับอนุญาตอาจผิดกฎหมาย

5 สแกนทุกแบบพร้อมกัน

หน้าเว็บ: ติ๊กทั้ง 4 แบบ แล้วกรอกทั้ง 3 ช่อง (โฟลเดอร์, image, URL) งานเดียวจะรันทุกเครื่องมือตามลำดับ แล้วออกรายงานฉบับเดียว

CLI: ไม่ต้องใส่ `--scans` ระบบจะเลือกให้จากสิ่งที่ใส่มา (`--repo` → SAST + Dependency, `--image` → Container, `--url` → DAST)

```
python run_pipeline.py --project NodeGoat `
  --repo C:\Allprojects\Targets\NodeGoat `
  --image nodegoat:latest `
  --url http://host.docker.internal:4000
```

ตัว `` ` `` ท้ายบรรทัดคือการขึ้นบรรทัดใหม่ใน PowerShell ถ้า copy แล้วมีปัญหา ให้เขียนรวมเป็นบรรทัดเดียว

ตัวอย่างผลสแกน NodeGoat	จำนวนช่องโหว่ที่เปิดอยู่
Container (trivy-container)	119
Dependency (trivy-dep)	92
DAST (zap)	91
SAST (semgrep)	35
รวม	**337** (วิกฤต 23, สูง 112, ปานกลาง 108, ต่ำ 60, ข้อมูล 34)

สรุป: ต้องใส่อะไร และใช้ได้ที่ไหน

แบบ	ต้องใส่	ตัวเลือก CLI	แบบ A Docker	แบบ B บนเครื่อง
SAST	โฟลเดอร์ซอร์สโค้ด	`--repo`	ได้	ได้
Dependency	โฟลเดอร์ซอร์สโค้ด	`--repo`	ได้	ได้
Container	ชื่อ image	`--image`	เฉพาะ image บน registry	**ได้ทั้งหมด**
DAST	URL ของเว็บที่รันอยู่	`--url`	**ไม่ได้**	**ได้ (เปิด Docker Desktop)**

6. อ่านผลและรายงาน

6.1 ไฟล์ที่ได้หลังสแกน

อยู่ในโฟลเดอร์ `reports\ชื่อโปรเจกต์_วันที่_เวลา\` ของโปรเจกต์ (ดาวน์โหลด PDF จากหน้าเว็บได้เลย)

ไฟล์	ใช้ทำอะไร
`security_report.pdf`	รายงานสำหรับพิมพ์หรือส่งต่อ
`dashboard.html`	เปิดในเบราว์เซอร์ กรองตามความรุนแรงและค้นหาได้ โดยไม่ต้องเปิดระบบ
`report.html`	รายงานฉบับเดียวกับ PDF ในรูปแบบ HTML



รายการช่องโหว่ทั้งหมด (เรียงตามความเสี่ยง)

หัวข้อ	ประเภทช่องโหว่	รายละเอียดทางเทคนิค	คำแนะนำทางเลือก	Risk Score	EPSS	ความรุนแรง	เครื่องมือ	OWASP Category
nodejs-y18n: prototype pollution vulnerability	ช่องโหว่ใน Library (Dependency)	[trivy-dep] Package: y18n เวอร์ชัน 3.2.1 (แก้ไขได้ที่เวอร์ชัน 3.2.2, 4.0.1, 5.0.5) | [trivy-container] พบใน Docker image: nodegoat:latest | Package: y18n เวอร์ชัน 3.2.1 (แก้ไขได้ที่เวอร์ชัน 3.2.2, 4.0.1, 5.0.5)	หากไม่สามารถ upgrade library 'y18n' (เวอร์ชัน 3.2.1) ได้ทันที ให้ป้องกันด้วยการ Object.freeze(Object.prototype) ตั้งแต่จุดเริ่มต้นของแอปพลิเคชัน หรือ validate/sanitize input ก่อนส่งเข้าฟังก์ชันที่ merge/extend object (เช่น mixin, merge, extend) โดยเฉพาะการปฏิเสธ key ที่ชื่อ __proto__, constructor, prototype	10.0	69.4%	high	trivy-container, trivy-dep	A03:2021 - Injection
zlib: heap-based buffer over-read and overflow in inflate() in inflate.c via a large gzip header extra field	ช่องโหว่ใน Container Image	[trivy-container] พบใน Docker image: nodegoat:latest | Package: zlib เวอร์ชัน 1.2.12-r0 (แก้ไขได้ที่เวอร์ชัน 1.2.12-r2)	นักพัฒนาควรเปลี่ยนมาใช้ฟังก์ชันคัดลอกที่รองรับการระบุขนาดความยาว แทนฟังก์ชันที่ไม่จำกัดขนาด เช่น เปลี่ยนจาก strcpy เป็น strncpy พร้อมทั้งตรวจสอบขอบเขตและขนาดของบัฟเฟอร์เมื่อเข้าถึงในลูปเพื่อป้องกันการเขียนเกิน	9.97	19.0%	critical	trivy-container	A06:2021 - Vulnerable and Outdated Components

ตัวอย่างหน้ารายการช่องโหว่ในรายงาน PDF

6.2 แต่ละคอลัมน์หมายถึงอะไร

คอลัมน์	ความหมาย
ความรุนแรง	critical (วิกฤต) / high (สูง) / medium (ปานกลาง) / low (ต่ำ) / info (ข้อมูล) ระบบแปลงระดับของแต่ละเครื่องมือให้เป็นมาตรฐานเดียวกัน
Risk score (0–10)	คะแนนที่ใช้**เรียงลำดับว่าควรแก้อะไรก่อน** เริ่มจากระดับความรุนแรง (หรือ CVSS ถ้ามี) แล้วบวกเพิ่มได้สูงสุด 3 คะแนนตาม EPSS
EPSS	โอกาสที่ช่องโหว่นี้จะถูกโจมตีจริงใน 30 วันข้างหน้า (จาก FIRST.org) เช่น 69.4% แปลว่ามีการโจมตีจริงอยู่มาก ควรแก้ก่อน มีเฉพาะช่องโหว่ที่มีเลข CVE
CWE	ประเภทของจุดอ่อน เช่น CWE-89 = SQL injection
OWASP Category	หมวดใน OWASP Top 10 2021 เช่น A03:2021 - Injection
รายละเอียดทางเทคนิค	ไฟล์และบรรทัด (SAST), package และเวอร์ชันที่แก้แล้ว (Dependency/Container), URL และ parameter (DAST)
คำแนะนำทางเลือก	วิธีแก้หรือวิธีลดความเสี่ยงระหว่างที่ยังแก้ไม่ได้

6.3 ควรแก้อะไรก่อน

1. เรียงตาม **risk score** (Dashboard เรียงให้แล้ว) ไม่ใช่ตามระดับความรุนแรงอย่างเดียว
2. ช่องโหว่ที่ **EPSS สูง** มีคนโจมตีอยู่จริง ให้ความสำคัญเป็นพิเศษ
3. Dependency / Container ที่มี "เวอร์ชันที่แก้แล้ว" แก้ได้ง่ายที่สุด แค่อัปเกรด
4. แก้แล้วสั่งสแกนซ้ำ ช่องโหว่ที่หายไปจะถูกเปลี่ยนเป็น `fixed` และหายจาก Dashboard

7. ใช้ผ่าน CLI

```
python run_pipeline.py --project <ชื่อ> [ตัวเลือก]
```

ตัวเลือก	ความหมาย
`--project` (จำเป็น)	ชื่อโปรเจกต์ ถ้ายังไม่มีจะสร้างให้
`--repo`	โฟลเดอร์ source code (SAST, Dependency)
`--image`	ชื่อ Docker image (Container)
`--url`	URL ของเว็บที่รันอยู่ (DAST)
`--scans`	เลือกแบบ คั่นด้วย comma เช่น `sast,dependency` (ไม่ใส่ = เลือกจากสิ่งที่ใส่มา)
`--fail-on <ระดับ>`	จบด้วย exit code 1 ถ้ายังมีช่องโหว่ระดับนี้ขึ้นไปเปิดอยู่ (critical, high, medium, low, info) ใช้ใน CI (บทที่ 8)
`--queue`	ใส่งานเข้าคิวให้ worker ทำ แล้วจบทันที (ใช้ร่วมกับ `--fail-on` ไม่ได้)
`--no-report`	สแกนอย่างเดียว ไม่สร้างรายงาน
`--no-epss`	ไม่ดึงคะแนน EPSS (ใช้ตอนไม่มีอินเทอร์เน็ต)
`--with-remediation`	ให้ Gemini เขียนคำแนะนำให้ CWE ที่ยังไม่มี (ต้องตั้ง `GEMINI_API_KEY` และ clone OWASP CheatSheetSeries ไว้ในโฟลเดอร์โปรเจกต์ ใช้เวลา 13–40 วินาทีต่อ CWE)
`--output-dir`	โฟลเดอร์เก็บรายงาน (ค่าเริ่มต้น: `reports` ในโฟลเดอร์โปรเจกต์)

ตัวอย่าง

```
# สแกนแล้วให้ worker ทำแทน (ไม่ต้องรอ)
python run_pipeline.py --project NodeGoat --repo C:\Allprojects\Targets\NodeGoat --queue

# สร้างรายงานใหม่จากข้อมูลเดิม โดยไม่สแกนซ้ำ
python generate_report.py --project NodeGoat

# ดูตัวเลือกทั้งหมด
python run_pipeline.py --help
```

8. ใช้ใน CI/CD (security gate)

ใส่ `--fail-on` แล้วคำสั่งจะจบด้วย **exit code 1** ถ้ายังมีช่องโหว่ที่ระดับนั้นขึ้นไปเปิดอยู่ ระบบ CI ทุกตัว (GitHub Actions, GitLab CI, Jenkins) จะถือว่า step นั้นไม่ผ่าน และหยุด build หรือ deploy ให้เอง

```
python run_pipeline.py --project MyApp --repo . --scans sast,dependency --fail-on critical
```

ค่า `--fail-on`	ไม่ผ่านเมื่อมีช่องโหว่
`critical`	วิกฤต (แนะนำสำหรับเริ่มต้น)
`high`	วิกฤต หรือ สูง
`medium`	ปานกลางขึ้นไป (เข้มงวด)

ตัวอย่าง GitHub Actions

VulnCompass ใช้วิธีนี้สแกนตัวเองทุกครั้งที่ push (ดูไฟล์เต็มได้ที่ `.github/workflows/security-scan.yml`) ขั้นตอนหลักคือ:

```
services:
  postgres:                     # ฐานข้อมูลชั่วคราว มีอยู่แค่ระหว่างรัน job
    image: postgres:16-alpine
    env: { POSTGRES_USER: ci, POSTGRES_PASSWORD: ci-only-throwaway, POSTGRES_DB: ci }
    ports: ["5432:5432"]

steps:
  - ติดตั้ง Python, ไลบรารีทำ PDF + ฟอนต์ไทย, requirements.txt
  - sudo bash scripts/install_trivy.sh       # Trivy แบบล็อกเวอร์ชันและตรวจ SHA-256
  - python init_db.py
  - python seed_cwe_mapping.py
  - python run_pipeline.py --project VulnCompass --repo . --scans sast,dependency --fail-on critical
  - อัปโหลดโฟลเดอร์ reports/ เป็น artifact (แม้ gate จะไม่ผ่าน)
```

> รายงาน PDF ของแต่ละรอบดาวน์โหลดได้จากหน้าผลการรันใน GitHub Actions หัวข้อ **Artifacts** (เก็บไว้ 14 วัน)

9. ใช้ผ่าน API

script หรือระบบอื่นเรียก API ได้โดยส่ง header `X-API-Key` ที่ตรงกับ `API_KEY` ใน `.env` (ถ้าไม่ได้ตั้ง `API_KEY` คำขอที่ใช้ key จะถูกปฏิเสธด้วย 503 ทั้งหมด ส่วนการ login ผ่านหน้าเว็บยังใช้ได้ตามปกติ) ดูรายละเอียดทุก endpoint ได้ที่ `http://127.0.0.1:8000/docs` (แบบ B)

Endpoint	ใช้ทำอะไร
`GET /health`	เช็กว่าระบบและฐานข้อมูลพร้อม (ไม่ต้องใช้ key)
`GET /projects`	รายชื่อโปรเจกต์และจำนวนช่องโหว่ที่เปิดอยู่
`GET /projects/{ชื่อ}/summary`	จำนวนแยกตามความรุนแรงและเครื่องมือ
`GET /projects/{ชื่อ}/findings`	รายการช่องโหว่ กรองด้วย `severity`, `tool`, `q`, `status`, `limit`
`POST /scans`	สั่งสแกน (ใส่คิว) ตอบกลับเลขงาน
`GET /scan-jobs/{เลขงาน}`	สถานะของงานและแต่ละเครื่องมือ
`GET /scan-jobs/{เลขงาน}/report`	ดาวน์โหลด PDF ของงานนั้น

ตัวอย่าง (PowerShell, แบบ B)

```
$h = @{ "X-API-Key" = $env:VC_API_KEY }      # ตั้งค่า key ไว้ในตัวแปร ไม่พิมพ์ลงในคำสั่งตรงๆ

# สั่งสแกน
$body = @{ project = "NodeGoat"; scans = @("sast","dependency");
           repo_path = "C:\Allprojects\Targets\NodeGoat" } | ConvertTo-Json
$job = Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/scans -Headers $h `
       -ContentType "application/json" -Body $body
$job.scan_job_id

# ดูสถานะ
Invoke-RestMethod -Uri "http://127.0.0.1:8000/scan-jobs/$($job.scan_job_id)" -Headers $h

# ดาวน์โหลดรายงานเมื่อเสร็จ
Invoke-WebRequest -Uri "http://127.0.0.1:8000/scan-jobs/$($job.scan_job_id)/report" -Headers $h -OutFile
report.pdf

# ช่องโหว่ระดับวิกฤต 20 อันดับแรก
Invoke-RestMethod -Uri "http://127.0.0.1:8000/projects/NodeGoat/findings?severity=critical&limit=20" -Headers
$h
```

ในแบบ A (Docker) API ไม่ได้เปิด port ออกมาตรงๆ ให้เรียกผ่าน `http://localhost:8080/api/...` แทน (เช่น `http://localhost:8080/api/scans`)

10. จัดการผู้ใช้และดูแลระบบ

10.1 จัดการผู้ใช้

แบบ B รันตรงๆ ส่วนแบบ A ให้ขึ้นต้นด้วย `docker compose exec api`

คำสั่ง	ทำอะไร
`python manage_users.py create <ชื่อ>`	สร้างผู้ใช้ (ถามรหัสผ่าน 2 ครั้ง อย่างน้อย 12 ตัวอักษร)
`python manage_users.py passwd <ชื่อ>`	เปลี่ยนรหัสผ่าน
`python manage_users.py disable <ชื่อ>`	ปิดบัญชี (login ไม่ได้ทันที)
`python manage_users.py enable <ชื่อ>`	เปิดบัญชีกลับ
`python manage_users.py list`	รายชื่อผู้ใช้ทั้งหมด

10.2 เปิด ปิด และอัปเดตระบบ (แบบ A)

คำสั่ง	ทำอะไร
`docker compose --profile app up -d`	เปิดระบบ
`docker compose --profile app stop`	ปิดระบบ (ข้อมูลอยู่ครบ)
`docker compose logs -f worker`	ดูการทำงานของ worker แบบสด (Ctrl+C เพื่อเลิกดู)
`git pull` แล้ว `docker compose --profile app up -d --build`	อัปเดตเป็นเวอร์ชันล่าสุด (ต้อง `--build` ทุกครั้งที่โค้ดเปลี่ยน)

> **ห้ามใช้ `docker compose down -v`** ตัว `-v` จะลบ volume ของฐานข้อมูล ข้อมูลการสแกนทั้งหมดจะหายถาวร

10.3 Backup ฐานข้อมูล

```
docker exec secpipeline-postgres pg_dump -U secpipeline -d secpipeline -Fc -f /tmp/secpipeline.dump
docker cp secpipeline-postgres:/tmp/secpipeline.dump backups\secpipeline_YYYYMMDD.dump
```

ใช้ `docker cp` ไม่ใช้ `>` ของ PowerShell เพราะจะทำให้ไฟล์ binary เสีย ไฟล์ backup มีข้อมูลซ่องโหว่ จึงต้องเก็บเป็นความลับ (โฟลเดอร์ `backups/` อยู่ใน `.gitignore` แล้ว)

11. แก้ปัญหาที่พบบ่อย

อาการ / ข้อความ	สาเหตุ	วิธีแก้
`repo_path` ไม่อยู่ในโฟลเดอร์ที่อนุญาต	โฟลเดอร์ไม่อยู่ใน allowlist หรือใช้ path ของ Windows ในแบบ Docker	แบบ A ใช้ `/targets/...` · แบบ B ตรวจ `SCAN_ALLOWED_REPO_ROOTS`
ไม่พบโฟลเดอร์ `repo_path`	พิมพ์ path ผิด หรือโฟลเดอร์ไม่ได้อยู่ใน `Targets`	ตรวจชื่อโฟลเดอร์ และตรวจว่าวางไว้ใต้ `Targets` จริง
`target_url` ไม่อยู่ในรายชื่อ `host` ที่อนุญาต	host ไม่อยู่ใน `SCAN_ALLOWED_HOSTS`	เพิ่ม host ใน `.env` แล้ว restart API
DAST: ไม่พบโปรแกรม `'docker'`	สั่ง DAST ในแบบ Docker	ใช้แบบ B
DAST ล้มเหลว / ไม่เจออะไร	เว็บเป้าหมายปิดอยู่ หรือใช้ `localhost`	เปิดเว็บก่อน และใช้ `http://host.docker.internal:<port>`
Container: หา image ไม่เจอ	image อยู่แค่ในเครื่อง แต่สแกนจากแบบ Docker	ใช้แบบ B หรือ push image ขึ้น registry
งานค้างที่ " ⏳ รอคิว"	worker ไม่ได้เปิด	แบบ B: `python worker.py` · แบบ A: `docker compose --profile app up -d`
หน้าเว็บ `ฐานข้อมูลไม่พร้อมใช้งาน` (503)	Docker Desktop หรือ PostgreSQL ปิดอยู่	เปิด Docker Desktop แล้ว `docker compose up -d` (ไม่ต้อง restart API)
`ระบบไม่พร้อมใช้งานชั่วคราว` (503)	Redis ปิดอยู่	เหมือนข้อบน
`login` ผิดหลายครั้งเกินไป (429)	ใส่รหัสผิด 5 ครั้งใน 15 นาที	รอ 15 นาที หรือเปลี่ยนรหัสด้วย `manage_users.py passwd`
หน้าเว็บ `502 Bad Gateway` (แบบ A)	API กำลังเริ่มทำงาน	รอ 5–10 วินาที แล้วกด F5
`set REDIS_PASSWORD in .env`	ยังไม่ได้ตั้งรหัสผ่าน Redis	เพิ่ม `REDIS_PASSWORD` ใน `.env`
Semgrep: `CERTIFICATE_VERIFY_FAILED` (แบบ B)	โปรแกรม antivirus (เช่น Norton) ดักการเชื่อมต่อ HTTPS	สร้าง `ca-bundle.pem` แล้วตั้ง `REQUESTS_CA_BUNDLE` และ `SSL_CERT_FILE` ใน `.env`
ภาษาไทยเป็นตัวอ่านไม่ออกใน PowerShell	console ไม่ได้ใช้ UTF-8	เปิดไฟล์ด้วย VS Code แทน

12. ใช้งานอย่างปลอดภัย

เรื่อง	สิ่งที่ต้องทำ
สแกนเฉพาะเป้าหมายที่ได้รับอนุญาต	โดยเฉพาะ DAST ที่ส่งคำขอไปยังเว็บจริง ทดสอบเฉพาะระบบของตัวเอง หรือมีหนังสืออนุญาต
allowlist ให้แคบที่สุด	`SCAN_ALLOWED_REPO_ROOTS` และ `SCAN_ALLOWED_HOSTS` ใส่เท่าที่ต้องใช้จริง
ไม่เปิดหน้าเว็บสู่อินเทอร์เน็ต	ระบบสั่ง server ไปสแกนเว็บอื่นได้ ถ้าเปิดสาธารณะอาจถูกใช้เป็นเครื่องมือโจมตีคนอื่น ถ้าต้องใช้ร่วมกันทั้งทีมให้เข้าผ่าน VPN
เก็บ `.env` เป็นความลับ	ไม่ commit ไม่ส่งในแชท ถ้าหลุดให้เปลี่ยนรหัสทุกตัวทันที
รายงานและ backup เป็นข้อมูลลับ	บอกจุดอ่อนของระบบไว้ชัดเจน ส่งเฉพาะคนที่เกี่ยวข้อง
อัปเดตสม่ำเสมอ	ช่องโหว่ใหม่ประกาศทุกวัน ทั้งในเป้าหมายและในเครื่องมือเอง ควร `git pull` และ build ใหม่เป็นประจำ
พบช่องโหว่ใน VulnCompass เอง	แจ้งแบบไม่เปิดเผยผ่านแท็บ **Security → Report a vulnerability** ของ repo (ดู `SECURITY.md`)

ขั้นตอนเริ่มใช้งานแบบย่อ

```
1. Copy-Item .env.example .env  แล้วเติมค่า (รวม REDIS_PASSWORD)
2. docker compose --profile app up -d --build
3. docker compose exec api python manage_users.py create admin
4. docker compose exec worker python run_pipeline.py --project NodeGoat --repo /targets/NodeGoat --scans sast
5. เปิด http://localhost:8080 → login → ดู Dashboard → หน้าการสแกน → สั่งสแกนรอบต่อไป → ดาวน์โหลดรายงาน
```